MEDHEAD

ARCHITECTURE DE SOLUTION

SOLUTION BUILDING BLOCKS

Catalogue des composants de solution pour la preuve de concept d'orientation hospitalière en situation d'urgence.

10
blocs de solution

06
blocs réalisés

04
blocs à consolider ou intégrer

PÉRIMÈTRE

API REST Spring Boot • interface Vue.js • données hospitalières simulées • calcul de trajet externe • tests • GitHub Actions • conteneurisation • composants d'intégration cible.

RÉFÉRENTIEL DE LIVRAISON

github.com/naegel23/P11-MedHead-PoC
github.com/naegel23/P11-MedHead-Architecture

Statut : démonstrateur technique autonome, sans connexion aux dossiers patients ni aux systèmes hospitaliers de production.

01 / PÉRIMÈTRE

Objet et règles de lecture

Les Solution Building Blocks (SBB) précisent les capacités nécessaires à la solution, leurs interfaces, leur maturité et leurs dépendances. Ils facilitent la réutilisation et l'extension de la preuve de concept MedHead.

Finalité métier

À partir d'une localisation d'urgence et d'une spécialité médicale, identifier les établissements correspondant aux critères, vérifier la disponibilité simulée de lits, puis présenter des propositions et des informations de trajet.

DISTINCTION DES PÉRIMÈTRES

PoC réalisée

Service hospitalier, base locale de démonstration, interface de consultation, validation technique, automatisation des builds et des tests.

Intégration à sécuriser

Quotas du fournisseur de routage, protection des accès, résilience et observabilité en conditions opérationnelles.

Architecture cible

API Gateway, bus d'événements, interopérabilité avec les domaines métiers et les systèmes réels de disponibilité hospitalière.

CONVENTIONS DE STATUT

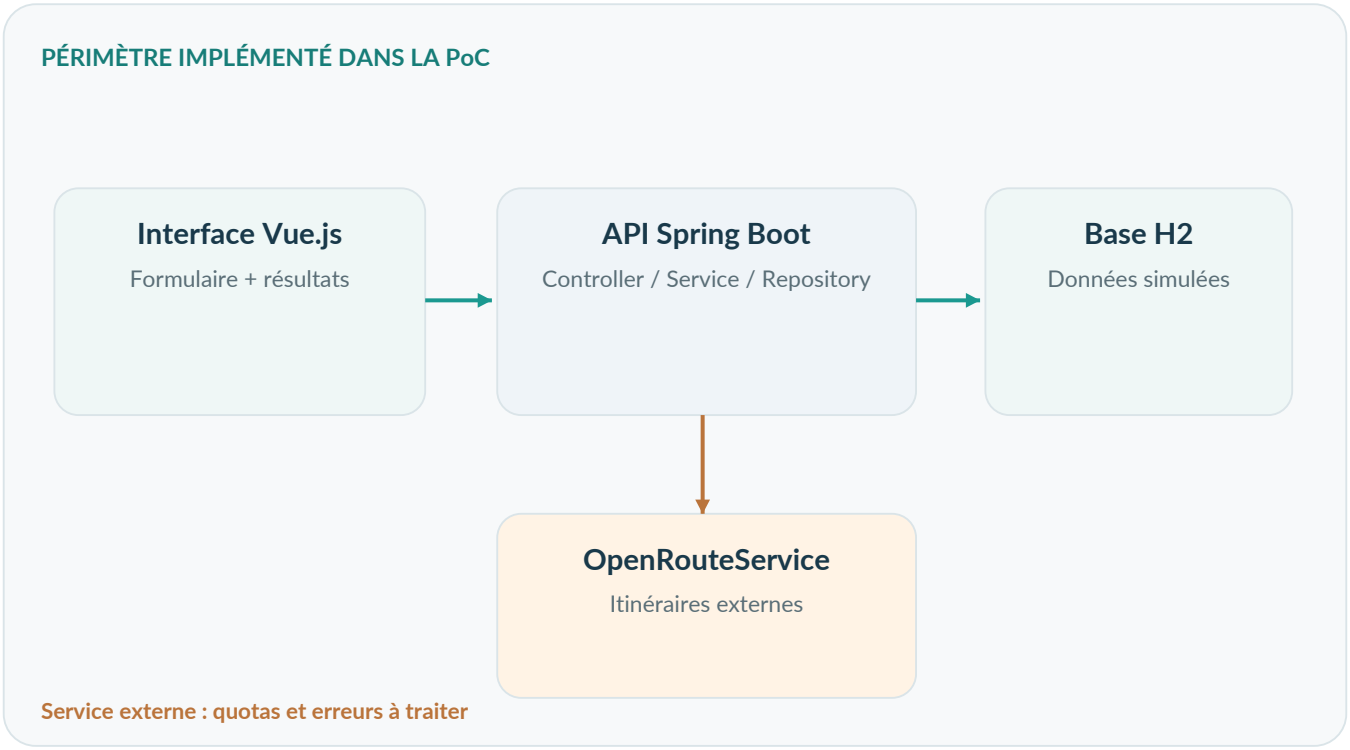
RÉALISÉ PARTIEL CIBLE

Le statut « réalisé » signifie que le bloc est présent et vérifié dans le périmètre expérimental. Il ne présume ni de son aptitude à la production ni de la validation des objectifs métier en exploitation.

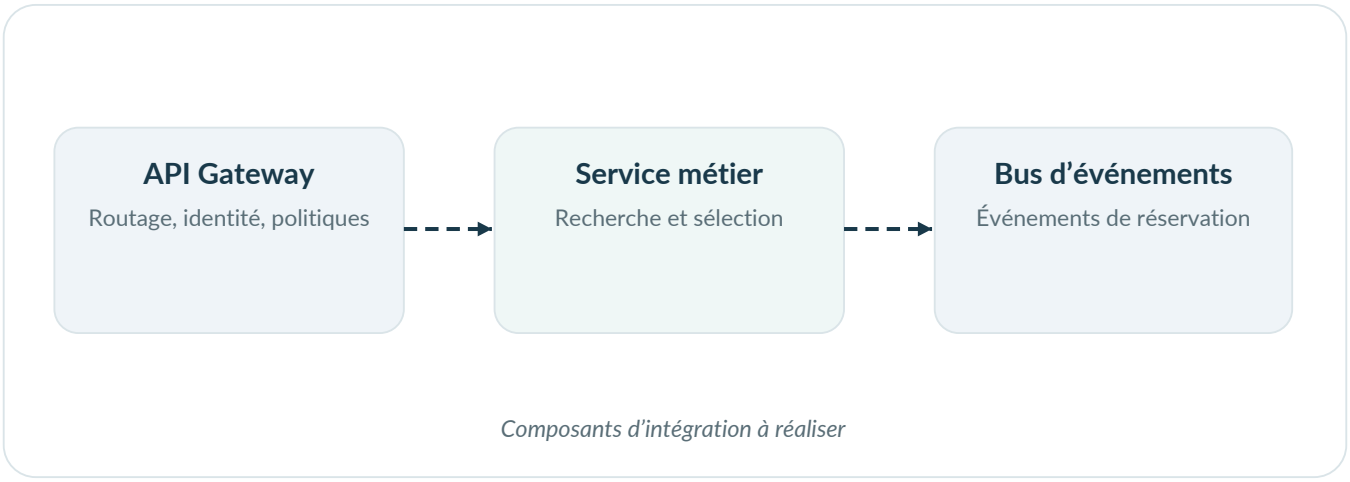
02 / VUE DE SOLUTION

Architecture des composants

La capacité de recommandation hospitalière est matérialisée par une API Spring Boot organisée en couches. L'architecture cible prévoit des briques d'intégration supplémentaires.



ÉVOLUTION DE L'ARCHITECTURE CIBLE



Décision architecturale : les couches internes du service assurent la séparation des responsabilités ; elles ne constituent pas à elles seules une plateforme microservices distribuée. Les composants en pointillés désignent des capacités d'intégration cible.

03 / INVENTAIRE

Catalogue et fiche SBB

Chaque bloc est identifié par une responsabilité explicite, les interfaces exposées, son état de réalisation, les travaux à conduire et les principes architecturaux auxquels il répond.

ID	BLOC DE SOLUTION	ÉTAT
SBB-01	Service d'orientation hospitalière	Réalisé
SBB-02	Architecture applicative multicouche	Réalisé
SBB-03	Données de test et persistance	Réalisé
SBB-04	Interface de consultation Vue.js	Réalisé
SBB-05	Plan de test et mesures de charge	Réalisé
SBB-06	Intégration continue et conteneurisation	Réalisé
SBB-07	Calcul d'itinéraires et résilience	Partiel
SBB-08	Sécurité et gestion des accès	À compléter
SBB-09	API Gateway	Cible
SBB-10	Bus d'événements métier	Cible

STRUCTURE DE LA FICHE

Fonction : responsabilité confiée au bloc. **Interfaces :** API, données, artefacts ou points d'intégration. **Réalisation :** éléments vérifiables du PoC. **Travaux à mener :** critères d'évolution. **Alignement :** principes métier, techniques et méthodologiques.

04 / CATALOGUE • BLOCS 01 À 02

Fiches de blocs de solution

Fonction, réalisation, interfaces, compléments et alignement des blocs SBB-01 et SBB-02.

SBB-01 Service de recherche et d'orientation

RÉALISÉ

FONCTION FOURNIE

Exposer la recherche des établissements, filtrer par spécialité, disponibilité de lits et distance, puis préparer la recommandation hospitalière.

INTERFACES / RÉALISATION

API HTTP/JSON Spring Boot : **/hospital**, filtres **/hospital/free/speciality/...** et recherche dans un rayon ; documentation OpenAPI sur **/v3/api-docs**.

TRAVAUX À MENER

Confirmer l'aptitude en production avec de véritables référentiels hospitaliers, une disponibilité fiable et une gestion contractuelle des erreurs.

ALIGNEMENT ARCHITECTURAL

B1 - continuité du service ; B2 - responsabilité claire ; C3 - contrats ouverts.

SBB-02 Structure applicative et conception multicouche

RÉALISÉ

FONCTION FOURNIE

Isoler les responsabilités de présentation REST, logique métier, accès aux données, modèles et gestion des exceptions.

INTERFACES / RÉALISATION

Java 17 / Spring Boot 3.5.16 : **HospitalController**, **HospitalService**, composants repository, entités hospitalières et gestion d'erreurs.

TRAVAUX À MENER

Formaliser les contrats d'intégration et les limites de contexte pour faciliter une future décomposition en services indépendants.

ALIGNEMENT ARCHITECTURAL

B2 - séparation des préoccupations ; C3 - standards d'architecture.

04 / CATALOGUE • BLOCS 03 À 04

Fiches de blocs de solution

Fonction, réalisation, interfaces, compléments et alignement des blocs SBB-03 et SBB-04.

SBB-03 Jeu de données hospitalières et persistance

RÉALISÉ

FONCTION FOURNIE

Fournir un référentiel hospitalier isolé afin de tester la recherche et le filtrage sans interaction avec des systèmes médicaux réels.

INTERFACES / RÉALISATION

Base embarquée **H2** alimentée par **data.sql** ; 1 290 établissements. Données d'établissement attribuées à une source NHS documentée ; spécialités et lits disponibles simulés.

TRAVAUX À MENER

Remplacer la simulation par une alimentation contrôlée, horodatée, auditable et vérifiable avant tout usage opérationnel.

ALIGNEMENT ARCHITECTURAL

A3 - maîtrise des données ; B1 - qualité de l'information ; C4 - isolement du PoC.

SBB-04 Interface de consultation des résultats

RÉALISÉ

FONCTION FOURNIE

Saisir la localisation, la spécialité et le rayon de recherche ; afficher les établissements et les informations de trajet fournies par l'API.

INTERFACES / RÉALISATION

Application **Vue.js 3 / Vite** ; appels REST/JSON vers le backend, tests frontend automatisés avec **Vitest** et **Vue Test Utils**.

TRAVAUX À MENER

Étendre les critères d'accessibilité, les cas d'erreur et les tests navigateur E2E automatisés sur un environnement représentatif.

ALIGNEMENT ARCHITECTURAL

A2 - valeur métier ; B2 - séparation client/service ; B4 - tests automatisés.

04 / CATALOGUE • BLOCS 05 À 06

Fiches de blocs de solution

Fonction, réalisation, interfaces, compléments et alignement des blocs SBB-05 et SBB-06.

SBB-05 Stratégie de tests et validation de charge

RÉALISÉ

FONCTION FOURNIE

Vérifier les comportements de l'API, l'intégration des composants et la tenue d'un scénario de recherche sous charge.

INTERFACES / RÉALISATION

JUnit 5, Maven, tests frontend et **Apache JMeter** ; 7 tests backend validés ; campagne de 100 000 requêtes, environ 998,93 req/s mesurées, 0 % d'erreurs.

TRAVAUX À MENER

Compléter par des tests de bout en bout automatisés, de résilience et de sécurité ; reproduire les mesures sur infrastructure cible, avec services externes.

ALIGNEMENT ARCHITECTURAL

B1 - fiabilité sous charge ; B4 - automatisation ; C4 - apprentissage mesuré.

SBB-06 Intégration continue et conteneurisation

RÉALISÉ

FONCTION FOURNIE

Automatiser les contrôles de qualité, les tests et la construction du backend et de l'interface, puis faciliter l'exécution reproductible du service.

INTERFACES / RÉALISATION

GitHub Actions sur push/PR de main ; Java 17, Maven, Node.js, tests backend/frontend et build ; conteneur Docker avec runtime Java 17.

TRAVAUX À MENER

Définir la publication des artefacts, les environnements de déploiement, les approbations, la traçabilité et le retour arrière avant toute CD de production.

ALIGNEMENT ARCHITECTURAL

B3 - intégration continue ; B4 - tests systématiques ; C2 - traçabilité.

04 / CATALOGUE • BLOCS 07 À 08

Fiches de blocs de solution

Fonction, réalisation, interfaces, compléments et alignement des blocs SBB-07 et SBB-08.

SBB-07 Routage externe et tolérance aux erreurs

PARTIEL

FONCTION FOURNIE

Obtenir des estimations de trajet pour départager ou présenter les hôpitaux compatibles selon le réseau routier.

INTERFACES / RÉALISATION

Intégration à **OpenRouteService** depuis le backend pour les calculs d'itinéraire ; dépendance HTTP et quotas de fournisseur.

TRAVAUX À MENER

Traiter les réponses 429 observées : temporisation, backoff, cache, circuit breaker, limitation du débit, supervision et éventuelle solution de repli.

ALIGNEMENT ARCHITECTURAL

B1 - continuité d'activité ; B2 - découplage des dépendances.

SBB-08 Sécurité, identité et protection des échanges

À COMPLÉTER

FONCTION FOURNIE

Protéger les accès aux services, contrôler les autorisations et sécuriser le transport, les secrets techniques et les traces.

INTERFACES / RÉALISATION

Le PoC traite des établissements et **aucune donnée nominative de patient** ; les contrôles de sécurité de production ne sont pas dans le périmètre livré.

TRAVAUX À MENER

Définir l'IAM (OIDC/OAuth 2.0), les rôles, HTTPS, gestion des secrets, journalisation sécurisée, revues de dépendances et tests de sécurité.

ALIGNEMENT ARCHITECTURAL

A3 - conformité ; B5 - sécurité dès la conception ; B1 - disponibilité.

04 / CATALOGUE • BLOCS 09 À 10

Fiches de blocs de solution

Fonction, réalisation, interfaces, compléments et alignement des blocs SBB-09 et SBB-10.

SBB-09 API Gateway et politiques d'accès

CIBLE

FONCTION FOURNIE

Centraliser l'entrée des API de la plateforme, le routage, l'application des politiques d'accès et le contrôle des flux.

INTERFACES / RÉALISATION

Point de terminaison cible en frontal des services ; contrats OpenAPI et règles de routage, d'authentification et de limitation de débit à définir.

TRAVAUX À MENER

Choisir une technologie, établir les contrats, configurer les politiques, les health checks, le monitoring et les tests d'intégration.

ALIGNEMENT ARCHITECTURAL

B2 - séparation des responsabilités ; B5 - contrôle d'accès ; C3 - interfaces standards.

SBB-10 Bus d'événements et publication métier

CIBLE

FONCTION FOURNIE

Découpler les services métiers via la publication d'événements, notamment pour les processus d'allocation ou de réservation de lits.

INTERFACES / RÉALISATION

Événements métier à spécifier : contrats, identifiants, clés d'idempotence, consommateurs, journalisation et garanties de livraison.

TRAVAUX À MENER

Choisir la technologie de messagerie, concevoir les schémas, le contrôle d'accès, les retries, la file d'échec et les mécanismes de rejeu.

ALIGNEMENT ARCHITECTURAL

B6 - capacités événementielles ; B1 - résilience ; B2 - faible couplage.

05 / COHÉRENCE ARCHITECTURALE

Traçabilité des principes

Les blocs apportent chacun une contribution mesurable à la plateforme. Les principes B1 à B6 constituent l'axe principal de validation technique.

B1	**Continuité des activités**
B2	**Séparation des responsabilités**
B3	**Intégration et livraison continues**
B4	**Tests automatisés**
B5	**Sécurité shift-left**
B6	**Événements métier**

CORRESPONDANCE PRINCIPES / BLOCS

B1	SBB-01, 03, 05, 07, 08, 10
B2	SBB-01, 02, 04, 07, 09, 10
B3	SBB-06
B4	SBB-04, 05, 06
B5	SBB-08, 09
B6	SBB-10

Lecture : un rattachement indique une contribution architecturale ou une exigence à satisfaire ; il ne signifie pas nécessairement que le principe est entièrement validé en exploitation.

06 / ÉVOLUTION MAÎTRISÉE

Dépendances et intégration

La livraison progressive des blocs limite le risque d'intégration et empêche la confusion entre démonstration fonctionnelle et exploitabilité clinique.

01

Socle du PoC

SBB-01 à SBB-06

API, application, jeu de données, tests, chaîne CI et exécution conteneurisée.

02

Fiabilisation des dépendances

SBB-07

Contrôle des quotas, réessais avec backoff, circuit breaker et instrumentation du routage.

03

Sécurisation des interfaces

SBB-08 à SBB-09

Contrats d'identité, authentification, permissions, secrets, TLS et politiques gateway.

04

Interopérabilité événementielle

SBB-10

Événements d'allocation / réservation, consommateurs, idempotence et garanties de diffusion.

05

Validation système

Tous les SBB

Essais représentatifs, disponibilité réelle, sécurité, résilience et mesure des KPI métier.

Point de contrôle : toute intégration à un système clinique devra établir la qualité, la fraîcheur et la responsabilité des données, les conditions de reprise, les accès autorisés et les conséquences d'une recommandation incorrecte.

07 / CONCLUSION ET DÉCISIONS

Critères de réception des blocs

Les critères ci-dessous servent à statuer sur la réutilisabilité des composants et sur les travaux nécessaires avant une intégration au système cible.

Comportement

Recherche, filtrage, retours API et consultation frontend vérifiables par scénarios de test.

Contrats

Interfaces REST décrites avec OpenAPI ; contrats d'événements définis pour les usages asynchrones.

Qualité logicielle

Tests automatisés reproductibles, traces CI consultables et analyse des échecs.

Performance

Mesures de latence et de débit sur un périmètre explicite ; cible 800 req/s et < 200 ms à reproduire avec les dépendances pertinentes.

Disponibilité

Gestion déterministe des timeouts, quotas, erreurs de routage et indisponibilités.

Sécurité

Authentification, autorisation, TLS et gestion des secrets documentées et testées.

Données et métier

Référentiels vérifiés ; évaluation opérationnelle des objectifs d'orientation et de temps de prise en charge.

CONCLUSION ARCHITECTURALE

Les blocs de service, de données, d'interface et de validation établissent la faisabilité technique de la PoC. Les composants de sécurité, de tolérance aux pannes et d'intégration événementielle constituent des prérequis à traiter pour l'architecture cible. Le PoC ne démontre pas, à lui seul, les KPI cliniques en exploitation.